



ML IoT

HCMUT EE MACHINE LEARNING & IOT LAB

HCMUT EE MACHINE LEARNING & IOT LAB

Bài tập về nhà — Buổi 1

Đại số tuyến tính cho AI

Môn học:

Python & Machine Learning

Học phần:

Module 1 — Toán học trong AI

Hạn nộp:

27/6/2026

Mục lục

1	Thông tin chung	2
1.1	Hình thức nộp	2
1.2	Công cụ	2
1.3	Lựa chọn dữ liệu	2
2	Bài 1 — Biểu diễn dữ liệu thành ma trận & độ tương đồng	4
3	Bài 2 — Biến đổi tuyến tính & SVD	5
3.1	Lựa chọn A — Ảnh	5
3.2	Lựa chọn B — Text	5
4	Nộp bài & Đánh giá	7
4.1	Cách nộp	7
4.2	Thang điểm tham khảo	7

1 Thông tin chung

Bài tập này củng cố các khái niệm nền tảng của Buổi 1: biểu diễn dữ liệu bằng vector và ma trận, đo độ tương đồng, biến đổi tuyến tính và phân rã SVD.

1.1 Hình thức nộp

- Nộp **một file notebook .ipynb** trong thư mục **week01/** của repo cá nhân (tạo từ repo mẫu trên GitHub).
- Notebook phải **chạy được từ đầu đến cuối** (Restart & Run All không lỗi).
- Mỗi phần kèm vài dòng **nhận xét** trong ô markdown.
- **Hạn nộp: 27/6/2026.**

1.2 Công cụ

Chỉ sử dụng **NumPy** và **Matplotlib** (nếu chọn ảnh, thêm **Pillow**). Không dùng thư viện Machine Learning cấp cao (scikit-learn, v.v.) cho phần tự cài đặt.

1.3 Lựa chọn dữ liệu

Chọn **một** trong hai loại dữ liệu và dùng xuyên suốt cả hai bài:

- **Text:** ít nhất 8 câu (tiếng Việt hoặc tiếng Anh) do bạn tự chọn.
- **Ảnh:** một ảnh của bạn (hoặc ít nhất 8 ảnh nhỏ), chuyển sang ảnh xám.

Gợi ý chuyển dữ liệu thành ma trận:

```
1 import numpy as np
2
3 # ===== Neu chon ANH =====
4 from PIL import Image
5 M = np.array(Image.open("anh.jpg").convert("L"), dtype=float) # ma
   tran pixel (H, W)
6 print(M.shape)
7
8 # ===== Neu chon TEXT (bag-of-words) =====
9 cau = ["cau thu nhat ...", "cau thu hai ..."] # >= 8 cau
```

```
10 vocab = sorted({w for s in cau for w in s.lower().split()})
11 def to_vector(s):
12     v = np.zeros(len(vocab))
13     for w in s.lower().split():
14         v[vocab.index(w)] += 1
15     return v
16 X = np.array([to_vector(s) for s in cau])           # (so_cau, so_tu)
17 print(X.shape)
```

2 Bài 1 — Biểu diễn dữ liệu thành ma trận & độ tương đồng

Mục tiêu: đưa dữ liệu thực tế về dạng vector/ma trận và đo độ tương đồng giữa các mẫu.

1. **Biến dữ liệu thành ma trận.** Tạo ma trận X : với text là (số câu $\times$ số từ); với ảnh là (số ảnh $\times H \cdot W$). In X .shape và giải thích mỗi hàng/cột đại diện cho gì.
2. **Phép toán cơ bản.** Tính vector trung bình theo cột, trừ trung bình (broadcasting); in shape trước/sau để minh họa quy tắc broadcasting.
3. **Cosine similarity.** Hoàn thiện hàm `cosine_similarity(X, Y=None)` cho batch (chuẩn hóa theo hàng, `keepdims=True`), trả về ma trận tương đồng.
4. **Truy vấn.** Viết hàm `search(query, top_k=3)` trả về `top_k` mục giống truy vấn nhất kèm điểm cosine.
5. **Nhận xét.** Cặp nào giống nhau nhất, cặp nào khác biệt nhất? Kết quả có khớp trực giác không?

```
1 def cosine_similarity(X, Y=None):
2     if Y is None:
3         Y = X
4     Xn = X / np.linalg.norm(X, axis=1, keepdims=True)
5     Yn = Y / np.linalg.norm(Y, axis=1, keepdims=True)
6     return Xn @ Yn.T                # ma tran tuong dong
7
8 # TODO: viet ham search(query, top_k=3) tra ve top_k muc giong nhat
```

3 Bài 2 — Biến đổi tuyến tính & SVD

Mục tiêu: hiểu ma trận như một phép biến đổi và dùng SVD để xử lý dữ liệu. Làm theo đúng loại dữ liệu bạn đã chọn.

3.1 Lựa chọn A — Ảnh

1. **Biến đổi tuyến tính.** Xây ma trận quay $R(\theta)$ và ma trận co giãn S ; áp lên tọa độ điểm (hoặc một hình đơn giản) và vẽ trước/sau bằng Matplotlib.
2. **Nén ảnh bằng SVD.** Phân rã $M = U\Sigma V^T$; giữ lại k giá trị kỳ dị lớn nhất (thử $k = 5, 20, 50$), tái tạo và hiển thị ảnh để so sánh.
3. **Đánh giá.** Tính tỉ lệ nén và sai số tái tạo theo từng k ; vẽ đồ thị sai số theo k .
4. **Nhận xét.** Với k nhỏ tới mức nào thì ảnh vẫn chấp nhận được? Liên hệ với giảm chiều và nén dữ liệu trong AI.

```
1 U, S, Vt = np.linalg.svd(M, full_matrices=False)
2
3 def reconstruct(k):
4     return (U[:, :k] * S[:k]) @ Vt[:, :]
5
6 # TODO: thu k = 5, 20, 50; hiện thị ảnh tái tạo
7 #       tính tỉ lệ nén và sai số tái tạo theo k; vẽ đồ thị sai số
```

3.2 Lựa chọn B — Text

1. Dùng ma trận X (câu $\times$ từ) từ Bài 1.
2. **Giảm chiều bằng SVD.** Dùng `np.linalg.svd` đưa mỗi câu về 2 chiều (LSA thu nhỏ).
3. **Trực quan hóa.** Vẽ scatter các câu trên mặt phẳng 2D, gắn nhãn từng câu.
4. **Nhận xét.** Các câu cùng chủ đề có nằm gần nhau không? Liên hệ với eigen/PCA và “trực chính” của dữ liệu.

```
1 Xc = X - X.mean(axis=0) # center du lieu
2 U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
3 coords = U[:, :2] * S[:2] # toa do 2D cua moi cau
4
```

```
5 # TODO: vẽ scatter coords và gan nhãn tung cau
```

4 Nộp bài & Đánh giá

4.1 Cách nộp

1. Đặt notebook .ipynb vào thư mục `week01/` trong repo cá nhân.
2. Commit và push lên GitHub.
3. Gửi link repo theo hướng dẫn của mentor trên Discord trước **27/6/2026**.

4.2 Thang điểm tham khảo

- Bài 1: 50% (code chạy đúng 40% · nhận xét 10%).
- Bài 2: 50% (code chạy đúng 40% · nhận xét 10%).
- **Bonus (+10%)**: với text, tự cài bộ phân loại 1-NN dựa trên cosine; với ảnh, vẽ đồ thị năng lượng tích lũy của các giá trị kỳ dị và chọn k giữ $\geq 90\%$ thông tin.

Chúc các bạn làm bài tốt! Mọi thắc mắc trao đổi tại kênh Discord của khóa học.